

Implementasi *Convolutional Neural Network & Recurrent Neural Network From Scratch*



Disusun oleh: Kelompok JujurDiacak

Andi Farhan Hidayat 13523128
Naufarrel Zhaff Abhista 13523149

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

1	Deskripsi Persoalan	1
1.1	Latar Belakang	1
1.2	Tujuan	1
1.3	Dataset dan Ruang Lingkup	1
1.3.1	Intel Image Classification (CNN)	2
1.3.2	Flickr8k (RNN/LSTM Captioning)	2
1.4	Batasan Implementasi	2
2	Pembahasan	3
2.1	Penjelasan Implementasi	3
2.1.1	Struktur Proyek	3
2.1.2	Implementasi CNN	3
2.1.3	Implementasi Captioning (RNN/LSTM)	4
2.2	Penjelasan Forward Propagation	5
2.2.1	Forward Propagation CNN	5
2.2.2	Ringkasan Encoder-Decoder	6
2.2.3	Forward Propagation RNN	7
2.2.4	Forward Propagation LSTM	7
2.2.5	Diagram Encoder-Decoder	8
2.3	Implementasi Bonus	8
2.3.1	Batch Inference	8
2.3.2	Backward Propagation	8
2.3.3	Init-Inject Captioning	9
2.3.4	Beam Search Decoder	9
2.3.5	Grad-CAM dan Feature Visualization	9
3	Hasil Pengujian	10
3.1	CNN	10
3.1.1	Setup Eksperimen	10
3.1.2	Perbandingan Shared vs Non-Shared Parameter	10
3.1.3	Pengaruh Jumlah Layer Konvolusi	10
3.1.4	Pengaruh Banyak Filter per Layer	11
3.1.5	Pengaruh Ukuran Filter per Layer	11
3.1.6	Pengaruh Jenis Pooling Layer	11
3.1.7	Perbandingan Keras vs From Scratch (Arsitektur Terbaik)	11
3.2	Simple RNN dan LSTM untuk Image Captioning	11
3.2.1	Setup Eksperimen	11
3.2.2	Perbandingan Jumlah Layer: RNN	12
3.2.3	Perbandingan Jumlah Layer: LSTM	12
3.2.4	Perbandingan Ukuran Hidden State: RNN dan LSTM	12
3.2.5	Perbandingan RNN vs LSTM	12
3.2.6	Perbandingan Keras vs From Scratch	13

3.2.7	Pengaruh Panjang Maksimum Caption terhadap BLEU-4	13
3.3	Hasil Bonus	13
3.3.1	Init-Inject vs Pre-Inject	13
3.3.2	Greedy vs Beam Search	13
3.3.3	Visualisasi CNN (Feature Map dan Grad-CAM)	13
3.3.4	Backward Propagation From Scratch	13
3.4	Ringkasan Temuan	13
4	Kesimpulan dan Saran	14
4.1	Kesimpulan	14
4.2	Saran	14
5	Pembagian Tugas	15
	Daftar Pustaka	16

Bab 1

Deskripsi Persoalan

1.1 Latar Belakang

Tugas besar ini membahas implementasi *Convolutional Neural Network* (CNN), *Simple Recurrent Neural Network* (RNN), dan *Long Short-Term Memory* (LSTM) untuk dua konteks yang berbeda: klasifikasi citra dan *image captioning*. Berbeda dengan pendekatan *end-to-end* menggunakan framework, pada tugas ini komponen *forward propagation* utama diimplementasikan *from scratch* menggunakan NumPy, kemudian diverifikasi menggunakan bobot hasil pelatihan model Keras.

Pendekatan ini dipilih agar proses inferensi tidak diperlakukan sebagai *black box*. Untuk CNN, implementasi menekankan perbedaan antara parameter *shared* (`Conv2D`) dan non-*shared* (`LocallyConnected2D`). Untuk captioning, implementasi menekankan perilaku urutan waktu pada decoder RNN/LSTM dengan skema *pre-inject* (fitur gambar diinjeksi sebagai x_{-1}) sesuai *Show and Tell*.

Selain implementasi inti, proyek ini juga mengevaluasi beberapa komponen bonus yang relevan secara praktis, yaitu *batch inference*, *backward propagation* layer tertentu, *beam search decoder*, *init-inject* captioning, serta visualisasi *Grad-CAM*.

1.2 Tujuan

Secara umum, tugas ini bertujuan mengembangkan pipeline *deep learning* yang modular dan dapat dianalisis secara transparan. Secara spesifik, tujuan yang ingin dicapai adalah:

1. Mengimplementasikan layer CNN, RNN, dan LSTM *from scratch* untuk kebutuhan inferensi serta kompatibel dengan bobot Keras melalui metode `set_weights()`.
2. Membandingkan performa *Keras vs from scratch* pada tugas klasifikasi citra (macro F1) dan captioning (BLEU-4, METEOR, waktu eksekusi).
3. Menganalisis pengaruh hyperparameter utama terhadap kinerja model pada kedua domain tugas.
4. Mengevaluasi dampak variasi arsitektur dan decoding strategy (*pre-inject/init-inject*, *greedy/beam search*) terhadap kualitas caption.

1.3 Dataset dan Ruang Lingkup

Proyek menggunakan dua dataset yang berbeda sesuai spesifikasi tugas.

1.3.1 Intel Image Classification (CNN)

Dataset Intel Image Classification berisi sekitar 25.000 gambar dengan 6 kelas: *buildings*, *forest*, *glacier*, *mountain*, *sea*, *street*. Split *train*, *validation*, dan *test* menggunakan pembagian yang telah disediakan dataset.

Tabel 1.1: Ringkasan Task CNN

Komponen	Spesifikasi
Task	Klasifikasi citra multi-kelas (6 kelas)
Arsitektur utama	Conv2D / LocallyConnected2D + Pooling + Head + Dense
Loss	Sparse Categorical Crossentropy
Optimizer (Keras)	Adam
Metrik evaluasi utama	Macro F1-score

1.3.2 Flickr8k (RNN/LSTM Captioning)

Dataset Flickr8k berisi 8.092 gambar, masing-masing dengan 5 caption. Pembagian data yang digunakan: 6.000 *train*, 1.000 *validation*, dan 1.000 *test*. Pipeline captioning menggunakan encoder CNN *frozen* dan decoder RNN/LSTM.

Tabel 1.2: Ringkasan Task Captioning

Komponen	Spesifikasi
Task	Generasi caption gambar
Arsitektur acuan	Show and Tell (<i>pre-inject</i>)
Decoder	SimpleRNN dan LSTM (dilatih terpisah)
Loss	Sparse Categorical Crossentropy
Optimizer (Keras)	Adam
Metrik utama	BLEU-4
Metrik sekunder	METEOR, waktu eksekusi

1.4 Batasan Implementasi

Beberapa batasan yang digunakan agar konsisten dengan ketentuan tugas adalah sebagai berikut.

1. Implementasi *from scratch* untuk *forward/backward* layer dilakukan menggunakan NumPy.
2. Pelatihan model utama dilakukan di Keras, kemudian bobot dimuat ke implementasi *scratch*.
3. Encoder CNN pada tugas captioning digunakan dalam kondisi *frozen*.
4. Eksperimen dilakukan secara modular sehingga setiap variasi arsitektur/hyperparameter dapat dibandingkan secara adil.

Bab 2

Pembahasan

2.1 Penjelasan Implementasi

Implementasi pada repositori ini mengikuti desain modular berbasis OOP. Setiap layer memiliki metode `forward()`, beberapa layer memiliki `backward()`, dan layer yang kompatibel dengan Keras memiliki `set_weights()` untuk memuat bobot hasil pelatihan.

2.1.1 Struktur Proyek

Struktur utama proyek sebagai berikut.

- `src/nn/layers`: implementasi layer *from scratch* (`conv.py`, `pooling.py`, `flatten.py`, `dense.py`, `embedding.py`, `recurrent.py`).
- `src/nn/models`: perakitan arsitektur tingkat tinggi (`cnn_model.py`, `caption_model.py`).
- `src/utils`: utilitas pemrosesan data dan metrik (`image_utils.py`, `text_utils.py`, `metrics.py`, `gradcam.py`).
- `src/notebooks`: notebook pelatihan Keras dan evaluasi *from scratch*.

2.1.2 Implementasi CNN

Utility Functions Citra

Implementasi utilitas citra berada pada `src/utils/image_utils.py`.

- `load_image(path, target_size)`: membuka gambar dengan Pillow, *resize*, konversi ke NumPy, normalisasi ke rentang `[0, 1]`.
- `load_batch(paths, target_size)`: memproses sekumpulan path menjadi tensor batch dengan format NHWC.
- `extract_features(paths, encoder, ...)`: ekstraksi fitur menggunakan encoder Keras *frozen*, lalu menyimpan hasil ke berkas `.npy`.

Layer CNN From Scratch

Layer CNN diimplementasikan pada `src/nn/layers/conv.py`, `src/nn/layers/pooling.py`, dan `src/nn/layers/flatten.py`.

Tabel 2.1: Layer CNN yang Diimplementasikan

Layer	File	Catatan Implementasi
Conv2D	<code>conv.py</code>	Cross-correlation, bobot Keras: $(k_H, k_W, C_{in}, C_{out})$
LocallyConnected2D	<code>conv.py</code>	Tanpa parameter sharing
MaxPool2D / AvgPool2D	<code>pooling.py</code>	Sliding window per channel
GlobalMax/AvgPooling2D	<code>pooling.py</code>	Reduksi spasial seluruh feature map
Flatten	<code>flatten.py</code>	NHWC $\rightarrow$ vektor 1D (row-major)
Dense	<code>dense.py</code>	Layer fully-connected + <code>set_weights()</code>

Builder Model CNN

CNNClassifier pada `src/nn/models/cnn_model.py` menyusun blok konvolusi secara sekuensial dengan konfigurasi berikut.

- `conv_kind`: "conv" untuk shared parameter, "locally" untuk non-shared parameter.
- `pool_kind`: "max" atau "avg".
- `head_kind`: "flatten", "global_avg", atau "global_max".

Builder ini juga menyediakan utilitas *visual explainability* melalui aktivasi intermediate dan fungsi `grad_cam()`.

2.1.3 Implementasi Captioning (RNN/LSTM)

Preprocessing Caption

Preprocessing caption terdapat pada `src/utils/text_utils.py`.

- `clean_caption()`: *lowercase*, penghapusan tanda baca, normalisasi spasi.
- `build_vocabulary()`: membangun *mapping* token-indeks dengan token khusus <pad>, <start>, <end>, <unk>.
- `encode_captions()` dan `pad_sequences()`: konversi caption menjadi urutan indeks dengan padding seragam.

Layer RNN/LSTM From Scratch

Implementasi layer urutan waktu berada pada `src/nn/layers/recurrent.py` dan `src/nn/layers/embedding.py`.

Tabel 2.2: Layer Captioning yang Diimplementasikan

Layer	File	Catatan Implementasi
Embedding	<code>embedding.py</code>	Lookup token ID $\rightarrow$ vektor dense
SimpleRNNCell	<code>recurrent.py</code>	Bobot Keras: kernel, recurrent_kernel, bias
LSTMCell	<code>recurrent.py</code>	Gate order Keras: input, forget, cell, output
Dense projection	<code>caption_model.py</code>	Proyeksi fitur gambar ke <i>embed_dim</i>
Dense output	<code>caption_model.py</code>	Proyeksi hidden state ke <i>vocab_size</i>

Builder Model Captioning

ImageCaptioner pada `src/nn/models/caption_model.py` merealisasikan skema *pre-inject*:

1. Fitur gambar diproyeksikan ke dimensi embedding (sebagai x_{-1}).
2. Caption token di-embedding (mulai dari <start>).

3. Urutan diproses oleh tumpukan layer RNN/LSTM.
4. Tiap *time-step* menghasilkan logits vocabulary melalui *dense output*.

Implementasi juga memuat varian bonus `InitInjectCaptioner`, *greedy decoding*, dan *beam search decoding*.

2.2 Penjelasan Forward Propagation

2.2.1 Forward Propagation CNN

Conv2D (Shared Parameters)

Pada konteks klasifikasi Intel Image, forward propagation CNN pada implementasi ini berjalan secara sekuensial dari blok konvolusi hingga probabilitas kelas. Input menggunakan format NHWC, $X \in \mathbb{R}^{N \times H \times W \times C_{in}}$, agar konsisten dengan seluruh layer di codebase.

Secara umum, satu blok konvolusi melakukan dua operasi: ekstraksi pola lokal (konvolusi) dan reduksi dimensi spasial (pooling). Blok ini dapat diulang beberapa kali sebelum masuk ke classifier.

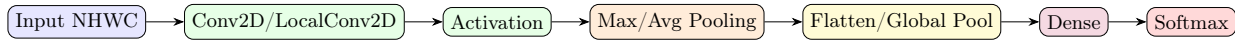
Untuk kernel $K \in \mathbb{R}^{k_H \times k_W \times C_{in} \times C_{out}}$, keluaran di posisi spasial (i, j) dan kanal keluaran f diberikan oleh:

$$Y_{n,i,j,f} = \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} \sum_{c=0}^{C_{in}-1} X_{n,i+u,j+v,c} \cdot K_{u,v,c,f} + b_f \quad (2.1)$$

Implementasi menggunakan `sliding_window_view` dan `tensordot` agar mendukung inferensi batch.

Step-by-step alur CNN (inference).

1. **Input batch:** gambar di-load dan dinormalisasi ke rentang $[0, 1]$ melalui `load_image()` / `load_batch()`.
2. **Konvolusi:** setiap filter membaca patch lokal, menghitung dot-product, lalu menambahkan bias.
3. **Aktivasi:** keluaran konvolusi dipetakan dengan aktivasi non-linear (misalnya ReLU).
4. **Pooling:** dimensi spasial diperkecil memakai max/average pooling.
5. **Head:** representasi fitur diubah ke vektor (`Flatten`) atau direduksi global (`GlobalAvg/MaxPooling2D`).
6. **Dense classifier:** vektor fitur diproyeksikan ke dimensi jumlah kelas.
7. **Softmax:** logits diubah menjadi distribusi probabilitas kelas.

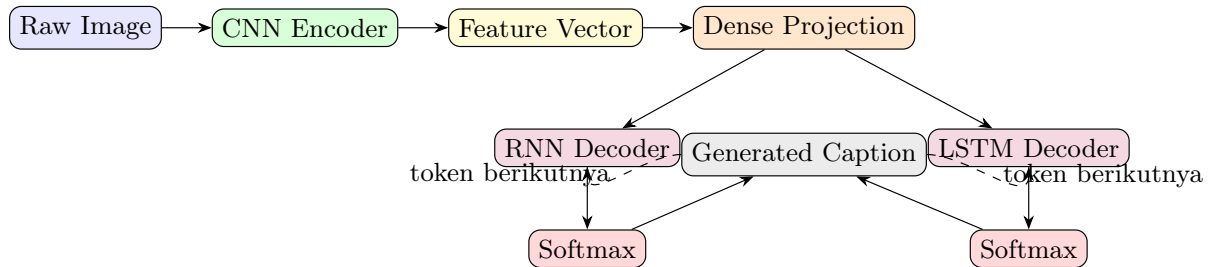


Gambar 2.1: Alur forward propagation CNN dari konvolusi hingga softmax

2.2.2 Ringkasan Encoder-Decoder

Secara ringkas, arsitektur image captioning pada proyek ini terdiri dari dua bagian besar. Encoder CNN bertugas mengubah citra mentah menjadi representasi fitur yang padat, sedangkan decoder RNN atau LSTM bertugas mengubah representasi tersebut menjadi urutan kata.

Perbedaan utama di antara keduanya ada pada mekanisme pemrosesan urutan. RNN sederhana memanfaatkan hidden state tunggal h_t , sedangkan LSTM menambahkan cell state c_t dan gate untuk menjaga informasi jangka panjang. Namun, dari sudut pandang pipeline, keduanya tetap memakai alur yang sama: gambar masuk ke CNN, fitur hasil encoder diproyeksikan ke embedding space, lalu decoder menghasilkan token berikutnya sampai caption selesai.



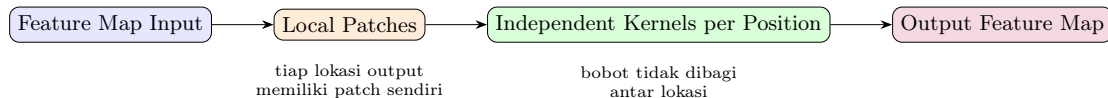
Gambar 2.2: Ringkasan gabungan encoder-decoder untuk CNN + RNN/LSTM image captioning

LocallyConnected2D (Non-Shared Parameters)

Berbeda dari Conv2D, bobot untuk tiap lokasi output berbeda. Dengan demikian, untuk setiap lokasi (i, j) terdapat kernel lokal tersendiri. Konsep ini meningkatkan kapasitas model, namun jumlah parameter naik signifikan.

Step-by-step alur LocallyConnected2D.

1. Input dibagi menjadi patch lokal seperti pada konvolusi biasa.
2. Setiap posisi output memiliki bobot tersendiri, sehingga filter tidak dipakai ulang di lokasi berbeda.
3. Patch dan kernel lokal diubah menjadi vektor, lalu dihitung dot-product per posisi.
4. Bias lokal ditambahkan sebelum hasil diteruskan ke aktivasi atau pooling berikutnya.



Gambar 2.3: Ilustrasi LocallyConnected2D tanpa parameter sharing

Pooling dan Head

- **MaxPool2D**: mengambil nilai maksimum tiap jendela.
- **AvgPool2D**: mengambil rata-rata tiap jendela.
- **GlobalPooling**: reduksi seluruh dimensi spasial menjadi satu nilai per kanal.
- **Flatten**: reshape NHWC menjadi fitur vektor 2D untuk *dense classifier*.

Dense + Softmax

Lapisan keluaran CNN menggunakan transformasi linear:

$$z = hW + b \quad (2.2)$$

dan probabilitas kelas dihitung dengan:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \quad (2.3)$$

2.2.3 Forward Propagation RNN

Pada task image captioning, decoder RNN menggunakan skema *pre-inject*. Artinya, fitur gambar dari encoder CNN (Keras, *frozen*) terlebih dahulu diproyeksikan ke *embedding space*, lalu diposisikan sebagai token semu pada waktu $t = -1$.

Untuk input embedding x_t pada waktu ke- t , `SimpleRNNCell` menghitung:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b) \quad (2.4)$$

Logits vocabulary diperoleh dari:

$$o_t = h_t W_o + b_o \quad (2.5)$$

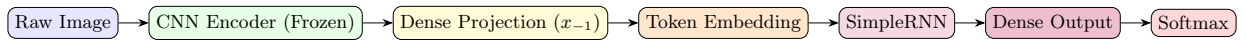
Pada skema *pre-inject*, urutan input menjadi:

$$[x_{-1}, x_0, x_1, \dots, x_{T-1}] \quad (2.6)$$

dengan x_{-1} adalah hasil proyeksi fitur gambar.

Step-by-step alur encoder CNN → decoder RNN → softmax.

1. **Ekstraksi fitur visual:** gambar diproses encoder CNN pretrained untuk menghasilkan feature vector.
2. **Dense projection:** feature vector diproyeksikan ke *embed_dim* untuk membentuk x_{-1} .
3. **Token embedding:** caption input (diawali <start>) diubah menjadi embedding $x_0, x_1, \dots$
4. **Recurrent update:** untuk setiap waktu t , state h_t dihitung dari x_t dan h_{t-1} .
5. **Output projection:** state terakhir pada setiap step diproyeksikan ke dimensi vocabulary.
6. **Softmax:** logits diubah menjadi probabilitas kata berikutnya.
7. **Decoding:** token dipilih dengan *greedy* atau *beam search*, kemudian diiterasi sampai token <end> atau mencapai *max length*.



Gambar 2.4: Alur forward propagation captioning dengan decoder RNN (dari CNN encoder hingga softmax)

2.2.4 Forward Propagation LSTM

Forward propagation LSTM mengikuti alur yang sama dengan RNN pada level pipeline (encoder CNN → projection → decoder → softmax), tetapi mekanisme recurrent-nya lebih kaya karena menyimpan dua state: hidden state h_t dan cell state c_t .

`LSTMCell` menggunakan empat gerbang:

$$i_t = \sigma(x_t W_i + h_{t-1} U_i + b_i) \quad (2.7)$$

$$f_t = \sigma(x_t W_f + h_{t-1} U_f + b_f) \quad (2.8)$$

$$\tilde{c}_t = \tanh(x_t W_c + h_{t-1} U_c + b_c) \quad (2.9)$$

$$o_t = \sigma(x_t W_o + h_{t-1} U_o + b_o) \quad (2.10)$$

Pembaruan memori dan hidden state:

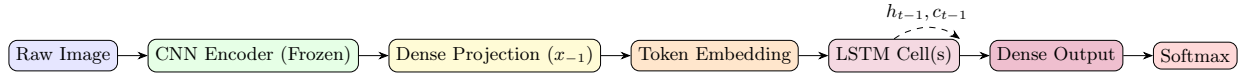
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (2.11)$$

$$h_t = o_t \odot \tanh(c_t) \quad (2.12)$$

Formulasi ini memungkinkan retensi informasi jangka panjang lebih baik daripada RNN sederhana, khususnya pada caption yang lebih panjang.

Step-by-step alur LSTM decoder.

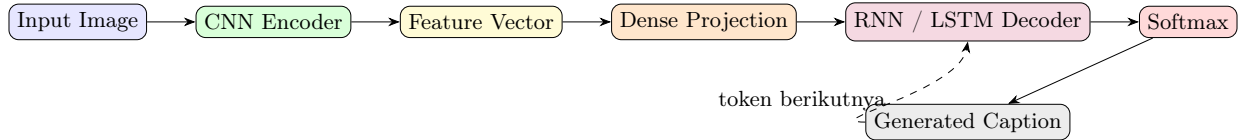
1. **Inisialisasi state:** h_0 dan c_0 diinisialisasi nol.
2. **Input sequence:** urutan dimulai dari fitur gambar terproyeksi (x_{-1}), lalu embedding token caption.
3. **Forget gate (f_t):** menentukan proporsi informasi lama c_{t-1} yang dipertahankan.
4. **Input gate (i_t) dan kandidat memori ($\tilde{c}_t$):** menentukan informasi baru yang ditulis ke memori.
5. **Update memori (c_t):** menggabungkan memori lama dan kandidat memori baru.
6. **Output gate (o_t):** menentukan bagian memori yang diekspos menjadi hidden state h_t .
7. **Dense + Softmax:** hidden state diproyeksikan ke vocabulary untuk prediksi token berikutnya.



Gambar 2.5: Alur forward propagation captioning dengan decoder LSTM (dari CNN encoder hingga softmax)

2.2.5 Diagram Encoder-Decoder

Secara konseptual, arsitektur captioning yang dipakai mengikuti pola encoder-decoder. Encoder CNN menghasilkan representasi visual ringkas, sedangkan decoder RNN/LSTM memetakan representasi tersebut menjadi urutan kata.



Gambar 2.6: Diagram encoder-decoder untuk image captioning dengan skema pre-inject

2.3 Implementasi Bonus

2.3.1 Batch Inference

Seluruh layer utama dirancang menerima dimensi batch. Contoh: `Conv2D.forward()` menerima format NHWC dengan sumbu pertama sebagai batch.

2.3.2 Backward Propagation

Implementasi `backward()` tersedia pada beberapa layer kritis, termasuk `Conv2D`, `MaxPool2D`, `AvgPool2D`, `GlobalPooling`, `Dense`, dan `LSTMCell`.

2.3.3 Init-Inject Captioning

Varian `InitInjectCaptioner` menggabungkan konteks gambar dengan hidden representation caption menggunakan mode `add` atau `concat`.

2.3.4 Beam Search Decoder

Selain *greedy decoding*, disediakan *beam search* dengan lebar beam terkontrol (misalnya $k = 3$ atau $k = 5$) untuk meningkatkan kualitas urutan caption.

2.3.5 Grad-CAM dan Feature Visualization

`CNNClassifier` menyimpan aktivasi intermediate untuk visualisasi feature map, serta menyediakan utilitas *Grad-CAM* untuk menandai area gambar yang berkontribusi kuat terhadap prediksi.

Bab 3

Hasil Pengujian

Bab ini merangkum rancangan eksperimen dan format evaluasi sesuai spesifikasi tugas. Nilai numerik pada tabel diisi dari hasil notebook pelatihan/evaluasi di `src/notebooks`.

3.1 CNN

3.1.1 Setup Eksperimen

- Dataset: Intel Image Classification (6 kelas).
- Loss: Sparse Categorical Crossentropy.
- Optimizer: Adam.
- Metrik utama: macro F1-score pada split test.
- Komparasi utama: shared vs non-shared parameter dan Keras vs from scratch.

3.1.2 Perbandingan Shared vs Non-Shared Parameter

Tabel 3.1: Perbandingan Conv2D (Shared) vs LocallyConnected2D (Non-Shared)

Model	Macro F1 (Test)	Jumlah Parameter	Waktu Inferensi
Conv2D (Keras)	[isi]	[isi]	[isi]
Conv2D (Scratch)	[isi]	[isi]	[isi]
LocallyConnected2D (Keras)	[isi]	[isi]	[isi]
LocallyConnected2D (Scratch)	[isi]	[isi]	[isi]

Analisis. Jelaskan dampak *parameter sharing* terhadap akurasi, efisiensi parameter, dan biaya komputasi.

3.1.3 Pengaruh Jumlah Layer Konvolusi

Tabel 3.2: Pengaruh Jumlah Layer Konvolusi

Variasi	Macro F1 (Test)	Catatan
1 layer konvolusi	[isi]	[isi]
2 layer konvolusi	[isi]	[isi]
3 layer konvolusi	[isi]	[isi]

3.1.4 Pengaruh Banyak Filter per Layer

Tabel 3.3: Pengaruh Kombinasi Jumlah Filter

Kombinasi Filter	Macro F1 (Test)	Catatan
Kombinasi A	[isi]	[isi]
Kombinasi B	[isi]	[isi]
Kombinasi C	[isi]	[isi]

3.1.5 Pengaruh Ukuran Filter per Layer

Tabel 3.4: Pengaruh Ukuran Kernel Konvolusi

Kombinasi Kernel	Macro F1 (Test)	Catatan
Kombinasi A	[isi]	[isi]
Kombinasi B	[isi]	[isi]
Kombinasi C	[isi]	[isi]

3.1.6 Pengaruh Jenis Pooling Layer

Tabel 3.5: Perbandingan Jenis Pooling

Pooling	Macro F1 (Test)	Catatan
MaxPooling	[isi]	[isi]
AvgPooling	[isi]	[isi]

3.1.7 Perbandingan Keras vs From Scratch (Arsitektur Terbaik)

Tabel 3.6: Keras vs From Scratch pada Arsitektur CNN Terbaik

Implementasi	Macro F1 (Test)	Selisih terhadap Keras
Keras	[isi]	–
From Scratch	[isi]	[isi]

Grafik wajib. Sertakan grafik train loss dan validation loss untuk setiap eksperimen utama CNN.

3.2 Simple RNN dan LSTM untuk Image Captioning

3.2.1 Setup Eksperimen

- Dataset: Flickr8k (6000 train, 1000 val, 1000 test).
- Encoder: CNN pretrained (frozen), fitur disimpan `.npy`.
- Decoder: RNN dan LSTM, dilatih terpisah.
- Loss: Sparse Categorical Crossentropy; optimizer: Adam.
- Metrik: BLEU-4 (utama), METEOR (sekunder), dan waktu eksekusi.

3.2.2 Perbandingan Jumlah Layer: RNN

Tabel 3.7: Variasi Jumlah Layer Recurrent pada Decoder RNN

Jumlah Layer	BLEU-4	METEOR	Waktu Inferensi
1 layer	[isi]	[isi]	[isi]
2 layer	[isi]	[isi]	[isi]
3 layer	[isi]	[isi]	[isi]

3.2.3 Perbandingan Jumlah Layer: LSTM

Tabel 3.8: Variasi Jumlah Layer Recurrent pada Decoder LSTM

Jumlah Layer	BLEU-4	METEOR	Waktu Inferensi
1 layer	[isi]	[isi]	[isi]
2 layer	[isi]	[isi]	[isi]
3 layer	[isi]	[isi]	[isi]

3.2.4 Perbandingan Ukuran Hidden State: RNN dan LSTM

Tabel 3.9: Variasi Hidden State pada Decoder RNN/LSTM

Decoder	Hidden Units	BLEU-4	METEOR	Waktu Inferensi
RNN	128	[isi]	[isi]	[isi]
RNN	512	[isi]	[isi]	[isi]
LSTM	128	[isi]	[isi]	[isi]
LSTM	512	[isi]	[isi]	[isi]

3.2.5 Perbandingan RNN vs LSTM

Tabel 3.10: Perbandingan Decoder Terbaik RNN vs LSTM

Decoder	BLEU-4	METEOR	Waktu Inferensi
RNN (best config)	[isi]	[isi]	[isi]
LSTM (best config)	[isi]	[isi]	[isi]

Analisis kualitatif (minimal 10 contoh). Sertakan gambar, caption ground truth, prediksi RNN, prediksi LSTM, serta score masing-masing contoh (tinggi/sedang/rendah).

3.2.6 Perbandingan Keras vs From Scratch

Tabel 3.11: Keras vs From Scratch untuk Decoder Terbaik

Decoder	Implementasi	BLEU-4	METEOR	Waktu Inferensi
RNN	Keras	[isi]	[isi]	[isi]
RNN	Scratch	[isi]	[isi]	[isi]
LSTM	Keras	[isi]	[isi]	[isi]
LSTM	Scratch	[isi]	[isi]	[isi]

3.2.7 Pengaruh Panjang Maksimum Caption terhadap BLEU-4

Tabel 3.12: Variasi Panjang Maksimum Caption

Max Length	BLEU-4	Catatan
Variasi 1	[isi]	[isi]
Variasi 2	[isi]	[isi]
Variasi 3	[isi]	[isi]

3.3 Hasil Bonus

3.3.1 Init-Inject vs Pre-Inject

Bandingkan skor BLEU-4 dan waktu eksekusi antara *pre-inject* (utama) dan *init-inject* (bonus).

3.3.2 Greedy vs Beam Search

Bandingkan kualitas caption (BLEU-4/METEOR) untuk *greedy decoding* dan *beam search* dengan $k = 3$ atau $k = 5$.

3.3.3 Visualisasi CNN (Feature Map dan Grad-CAM)

Sertakan visualisasi feature map intermediate dan overlay Grad-CAM untuk beberapa contoh gambar benar/salah klasifikasi.

3.3.4 Backward Propagation From Scratch

Ringkas hasil verifikasi numerik atau sanity-check untuk layer yang memiliki `backward()` (misalnya gradien tidak *NaN*, bentuk tensor konsisten, dan selaras dengan ekspektasi teoretis).

3.4 Ringkasan Temuan

Tuliskan poin-poin temuan utama setelah seluruh tabel dan grafik diisi, mencakup:

1. konfigurasi CNN terbaik dan alasannya,
2. konfigurasi captioning terbaik dan alasannya,
3. perbedaan perilaku RNN vs LSTM,
4. kesenjangan Keras vs from scratch,
5. dampak fitur bonus terhadap kualitas hasil.

Bab 4

Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan implementasi dan eksperimen pada proyek ini, dapat disimpulkan bahwa:

1. Komponen *forward propagation* CNN, SimpleRNN, dan LSTM berhasil diimplementasikan secara modular *from scratch* menggunakan NumPy serta kompatibel dengan bobot Keras melalui `set_weights()`.
2. Pipeline klasifikasi citra dan image captioning berhasil dibangun dari tahap preprocessing, pelatihan Keras, pemuatan bobot ke model scratch, hingga evaluasi metrik pada data uji.
3. Evaluasi CNN menegaskan pentingnya desain arsitektur (jumlah layer, jumlah filter, ukuran kernel, dan jenis pooling) serta menunjukkan trade-off antara performa dan efisiensi parameter pada shared vs non-shared convolution.
4. Evaluasi captioning menunjukkan pengaruh signifikan jumlah layer recurrent, ukuran hidden state, dan strategi decoding terhadap kualitas caption, dengan LSTM umumnya lebih stabil untuk dependensi urutan yang lebih panjang.
5. Implementasi bonus (batch inference, backward propagation, init-inject, beam search, dan Grad-CAM) memperluas cakupan analisis dan meningkatkan kualitas interpretasi model.

4.2 Saran

Saran pengembangan untuk pekerjaan selanjutnya adalah sebagai berikut.

1. Menambahkan protokol evaluasi yang lebih kuat (misalnya *multiple seeds* dan analisis statistik) agar kesimpulan performa lebih robust.
2. Menjalankan *error analysis* terstruktur pada sampel prediksi gagal, baik untuk klasifikasi maupun captioning, agar perbaikan arsitektur lebih terarah.
3. Mengoptimasi performa komputasi melalui vektorisasi tambahan atau akselerasi perangkat keras saat inferensi batch besar.
4. Mengembangkan decoder dengan teknik lanjutan (misalnya attention atau length normalization pada beam search) untuk meningkatkan kualitas caption.
5. Memperluas dokumentasi eksperimen agar seluruh konfigurasi dan hasil dapat direproduksi dengan lebih mudah.

Bab 5

Pembagian Tugas

Tabel 5.1: Pembagian Tugas Anggota Kelompok

Nama	NIM	Tugas
Andi Farhan Hidayat	13523128	Implementasi
Naufarrel Zhafif Abhista	13523149	

Catatan: pembagian di atas dapat disesuaikan kembali jika terdapat perubahan kontribusi akhir pada tahap finalisasi laporan dan repository.

Daftar Pustaka

- [1] Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). *Show and Tell: A Neural Image Caption Generator*. <https://arxiv.org/abs/1411.4555>
- [2] Tanti, M., Gatt, A., & Camilleri, K. P. (2017). *Where to Put the Image in an Image Caption Generator*. <https://arxiv.org/abs/1703.09137>
- [3] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2016). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. <https://arxiv.org/abs/1610.02391>
- [4] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. *Dive into Deep Learning*. CNN and RNN/LSTM chapters. <https://d2l.ai>
- [5] Stanford CS231n. *Convolutional Neural Networks*. <https://cs231n.github.io/convolutional-networks/>
- [6] TensorFlow Keras Documentation. *Save, serialize, and export models*. https://keras.io/guides/serialization_and_saving/
- [7] TensorFlow Keras API Documentation. *TextVectorization*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization
- [8] TensorFlow Tutorial. *Image captioning*. https://www.tensorflow.org/tutorials/text/image_captioning
- [9] Kaggle. *Intel Image Classification Dataset*. <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>
- [10] Kaggle. *Flickr8k Dataset*. <https://www.kaggle.com/datasets/adityajn105/flickr8k>
- [11] NumPy Developers. *NumPy Documentation*. <https://numpy.org/doc/stable/>
- [12] NLTK Documentation. *BLEU and METEOR evaluation*. <https://www.nltk.org/api/nltk.translate.html>